BIGDATA-N14

Kiến trúc và kiến thức nền dự án

1. Mục tiêu và phạm vi

BIGDATA-N14 chuyển một môi trường Hadoop/Hive/Spark đã thử trên một máy vật lý sang cụm gồm một Master và bốn Worker, mỗi máy Windows 11 chạy Docker Desktop Linux containers; Tailscale cung cấp mạng overlay giữa các máy. Không yêu cầu cài Hadoop/Java trực tiếp lên Windows hoặc chuyển host sang Ubuntu.

Việc cần chứng minh cuối cùng không chỉ là container khởi động hay bài toán trả kết quả đúng, mà là các Worker đăng ký vào HDFS/YARN và Spark application thực sự sử dụng tài nguyên của cluster.

2. Kiến trúc logic

Thành phần	Vai trò trong dự án	Cách hiểu ngắn gọn
Docker image	Đóng gói Java, Hadoop, Hive, Spark, cấu hình và entrypoint.	Cùng tag giúp các máy chạy cùng bộ phần mềm; không chứa trạng thái dữ liệu runtime nên không thay thế HDFS volume.
Master container	NameNode, ResourceManager, Hive Metastore/HiveServer2 theo entrypoint; điểm submit Spark client mode.	Điểm quản lý namespace HDFS, điều phối YARN và cung cấp dịch vụ điều khiển/truy vấn.
Mỗi Worker container	DataNode + NodeManager theo cấu hình hiện tại.	Lưu block HDFS và chạy container/tác vụ được YARN cấp phát.
Tailscale	Kết nối các máy vật lý qua IP tailnet.	Là đường đi mạng; tự nó không cấu hình hostname Hadoop, không tự mở firewall và không đảm bảo các cổng dịch vụ đang nghe.
Compose + .env	Khởi tạo container và truyền role/IP/hostname của node.	Compose chỉ diễn giải biến; phải xem `config` đã render để chắc biến được nạp đúng.
Docker volumes	Lưu metadata NameNode, dữ liệu DataNode và metadata Hive ngoài vòng đời container.	Xóa/tạo lại container không đồng nghĩa dữ liệu được xóa; ngược lại, volume mới là trạng thái mới, không tự mang dữ liệu local.

3. Các thành phần phần mềm đang đóng gói

Hadoop HDFS: Hệ thống tệp phân tán. NameNode quản lý metadata/namespace và vị trí block; DataNode lưu block thực tế. `hdfs dfsadmin -report` cho biết DataNode đang sống. Số container đang chạy không thay thế được kiểm tra này.

Hadoop YARN: Lớp quản lý tài nguyên và thực thi ứng dụng. ResourceManager điều phối tài nguyên; NodeManager ở mỗi Worker quản lý container trên node đó. `yarn node -list -all` cho thấy các NodeManager đã đăng ký và trạng thái của chúng.

Apache Spark: Engine xử lý phân tán được dùng cho các bài phân tích. `spark-submit` gửi ứng dụng theo cấu hình Spark; cấu hình hiện tại dùng `spark.master yarn` và `spark.submit.deployMode client`.

PySpark: API Python để viết ứng dụng Spark. Các file `word_count.py`, `word_mean.py`, `word_median.py`, `word_std_dev.py` là chương trình ứng dụng; PySpark không phải scheduler thay thế YARN.

Apache Hive: Lớp dữ liệu dạng bảng và truy vấn SQL. Metastore lưu metadata bảng; HiveServer2 là giao diện truy vấn theo cấu hình. Hive không đồng nghĩa với HDFS: file thực tế có thể nằm ở đường dẫn HDFS, còn bảng mô tả cách đọc/diễn giải file đó.

Java/JVM: Môi trường chạy Hadoop, Hive và phần lớn thành phần Spark. Image hiện khai báo Java 8; Hadoop 3.4.0, Hive 3.1.3, Spark 3.5.1 theo metadata của image.

4. Luồng một bài Spark trên cluster

1. Dữ liệu được đặt trong HDFS ở đường dẫn input mà script sử dụng, ví dụ `/nhom14/input/word/` hoặc `/nhom14/input/hive_sales/`.
2. Người dùng chạy `spark-submit` từ Master. Vì driver đang để client mode, tiến trình driver nằm ở phía client nơi submit, còn việc phân bổ task/executor phụ thuộc cấu hình và tài nguyên YARN.
3. Spark yêu cầu YARN cấp phát tài nguyên; NodeManager trên các Worker quản lý container thực thi được cấp.
4. Các task đọc partition dữ liệu, tính toán và trả kết quả/log. HDFS DataNode lưu block; YARN UI và log dùng để kiểm tra node/tác vụ thực sự tham gia.
5. Kết quả đúng là điều kiện cần, nhưng chưa đủ để chứng minh phân tán. Cần bằng chứng về application, task/executor/container và node trong YARN.

5. Môi trường local và môi trường 5 máy khác nhau

Chủ đề	Local đã thử	Cluster 5 máy
Mạng	Các container chia sẻ network của cùng Docker Engine; bridge/DNS nội bộ có thể hoạt động.	Mỗi máy có Docker Engine riêng. IP bridge của container cục bộ không phải địa chỉ node từ xa; cần Tailscale IP/hostname và TCP port truy cập được.
Biến môi trường	Có thể từng dùng Compose và .env local riêng.	Mỗi máy cần .env đúng vai trò; các biến chung phải nhất quán và `compose config` không có IP/hostname rỗng.
HDFS state	Volume local riêng có thể đã được format và chứa dữ liệu.	Volume mới có metadata mới. Không format nếu không chắc volume là mới/rỗng. Dataset local phải được nạp lại vào HDFS cluster.
Bằng chứng	Kết quả thành công xác nhận chương trình hoạt động trên môi trường local.	Cần DataNode live, NodeManager RUNNING và bằng chứng job dùng tài nguyên cluster.

6. Cấu hình quan trọng đã thấy trong dự án

HDFS: `fs.defaultFS=hdfs://nghia-master:9000`; NameNode RPC cổng 9000, giao diện NameNode 9870; replication được đặt là 3 theo file hướng dẫn nguồn.

YARN: ResourceManager hostname `nghia-master`; các cổng 8030–8033 và giao diện web 8088 đã được nêu trong hướng dẫn. DataNode/NodeManager còn có các cổng dịch vụ bổ sung như 9864/9866/9867 và 8040–8042.

Hive: Các cổng được nêu trong image/config gồm 9083, 10000 và 10002. Hive table `default.online_retail` đã được thử với CSV external table/OpenCSVSerde, bỏ qua một dòng header; các cột có thể được đọc dạng chuỗi và cần CAST tường minh khi tính số.

Spark: ` spark.master yarn`, `spark.submit.deployMode client`, `spark.executor.instances 4`, driver/executor/YARN AM memory 1g theo `spark-defaults.conf` đã cung cấp. Số executor cấu hình không đảm bảo có đủ bốn Worker hoặc mọi executor chạy trên node khác nhau.

HDFS data paths: File word: `/nhom14/input/word/description.txt`; CSV: `/nhom14/input/hive_sales/online_retail.csv` theo các đường dẫn được tài liệu nêu. Xác nhận thực tế bằng `hdfs dfs -ls` sau khi upload.

7. Mạng, hostname và Docker Desktop

Compose hiện dùng `network_mode: host`. Trên Docker Desktop, hành vi host networking phụ thuộc cấu hình/phiên bản Docker Desktop và không nên mặc định coi Linux container chia sẻ đầy đủ giao diện mạng Tailscale của Windows như Docker Engine native trên Linux. Cần thử kết nối TCP giữa máy với đúng cổng dịch vụ và kiểm tra hostname phân giải đúng; `tailscale ping` chỉ chứng minh đường kết nối Tailscale ở mức ping, không chứng minh HDFS/YARN hoạt động.

Lệnh `docker compose ... config` của nhóm vừa cung cấp đã cảnh báo các biến MASTER_IP, WORKER1-4_IP và WORKER1-4_HOSTNAME rỗng; worker config cũng có NODE_HOSTNAME, WORKER_LOGICAL_NAME và WORKER_IP rỗng. Đây là bằng chứng lệnh được chạy chưa nạp các biến cần thiết. Nếu `.env` ở thư mục gốc D:\BigData-N14, kiểm tra bằng `--env-file .\.env` như sau:

```
docker compose --env-file .\.env -f .\docker\docker-compose.master.yml config
docker compose --env-file .\.env -f .\docker\docker-compose.worker.yml config
```

Nếu các lệnh trên vẫn ra rỗng, xác minh file `.env` đang ở đúng thư mục, tên file không bị thành `.env.txt`, và biến trong file khớp đúng tên mà Compose dùng. `config` là bước xem cấu hình được render, không khởi động container.

8. Lệnh xác minh cốt lõi

```
# Chạy trên Master: tình trạng DataNode
docker exec -u hadoopnghia nghia-master hdfs dfsadmin -report

# Chạy trên Master: tình trạng NodeManager
docker exec -u hadoopnghia nghia-master yarn node -list -all

# Kiểm tra alias hostname trong container Master
docker exec nghia-master getent hosts nghia-master
docker exec nghia-master getent hosts khang-worker
docker exec nghia-master getent hosts trankhoa-worker
docker exec nghia-master getent hosts hongkhoa-worker
docker exec nghia-master getent hosts luan-worker
```

Mục tiêu cuối cùng là 4 Live DataNode và 4 NodeManager RUNNING khi đủ bốn Worker, sau đó kiểm tra application trên ResourceManager UI: http://100.75.213.56:8088 (chỉ dùng nếu IP và cổng đã xác nhận thực tế).

9. Thuật ngữ cần nhớ

Thuật ngữ	Ý nghĩa trong dự án
Image	Mẫu filesystem + cấu hình + metadata chạy container; không phải dữ liệu HDFS đang chạy.
Container	Tiến trình/instance được tạo từ image; mỗi host chạy container riêng.
Volume	Lưu trạng thái bền vững như metadata NameNode/DataNode/Metastore.
NameNode / DataNode	Quản lý metadata HDFS / lưu block dữ liệu.
ResourceManager / NodeManager	Điều phối tài nguyên YARN / quản lý thực thi trên từng Worker.
Driver / Executor	Driver điều phối ứng dụng Spark / executor chạy các task.
Metastore	Lưu metadata bảng Hive; không phải nơi mặc định lưu toàn bộ CSV.
Checksum SHA256	Dấu kiểm tra để xác nhận file TAR nhận được trùng với file Master phát hành.

Nguồn nội dung: tài liệu triển khai Cluster do nhóm cung cấp và thông tin `docker image inspect` / `docker compose config` trong trao đổi. Những điểm cần xác minh thực địa được đánh dấu là điều kiện kiểm tra, không được xem là kết quả đã đạt.